

Done by: Aashish Chaudhary (3/24/26)

Covers: Package management SysAdmin

TASK: Compile and Install from A source

----->

Metaphor:

Recipe -----> Kitchen Tools(Oven, Knife, eggs, flour) ---> Cake ----> Kids Eat it

Source Code (Written in C or C++, or Java, etc) --> Build Tools ---> Compile and Install -> Computer runs Executable (Computer understands only Binary)

Things to Do:

1. Download the Source PKG (note: only download not install)
2. Uncompress the compressed file.
3. Note the Required Dependencies from ReadMe.md file
4. Install dependencies, and build tools from the file
5. Make the .make file and install the pkg.

Boom! It installs. It's simple as that. However, it is smooth as if everything goes well. There are some circumstances, you might run into like me. Let me tell you what happened. First, let me tell you what I did.

In my project. I had already downloaded and uncompressed the source package for the tool, Fail2Ban. The Readme file has listed 'python setup tools and python version' information for its build tools. I simply followed those instructions and installed python setup tools. My machine had already preinstalled a python on it. So, I simply ignored installing the Python step.

Fail2ban is written in Python. I did not need to compile it. I simply had to run setup.py that was in uncompressed directory. When I ran "sudo python /path/to/uncompressed/files/setup.py install", it ran into the problem. The

pre-installed python3 in my machine could not read Fail2ban's source code. It required a python2 version less than 3, specifically 2.7 or something to be exact to install the package. Here, the ignoring step of installing python gives me some headaches.

Now, the next Step was to install python2.7. I downloaded from Python's official website running "`curl -O /link/to/python/archive`", again downloading the source package. Why? Because, the repository did not have python2 in it. It was already obsolete. One more thing, I had to make sure the newer package does not overwrite the existing python system packages. That is why I chose to make a directory inside `/usr/local/`. This would help me uninstall and clean up everything easily. I configured the path `/usr/local/python2` for '`./configure --prefix=/usr/local/python2`'. This will put everything in this directory. One more thing, if we do not configure with prefix option, it scatters the files all over to the multiple directories.

Another step is the actual compile; the command is: "**`make -j $(nproc)`**". Don't worry, you don't have to know what it means. I also googled it. All I know from what I read at the moment, is, it simply compiles with whatever **number of processes (i.e. n) you** have. It speeds up the thing. After a minute or two, once "**`.make`**" file is ready. You simply run '**`sudo make install`**'. What does it do? It copies the binaries to wherever we just configured in the '`./configure --prefix=/usr/local/python2`' command.

Now, Our python2 is successfully installed. Once the python was installed in the system. We simply use that python to install the fail2ban. Here's the command I had initially tried to run and it did not run because I did not have correct python for the package. "**`sudo /usr/local/bin/python2.7 setup.py install`**".

Hence, in short, I installed fail2ban from the source package to my machine. I initially uncompressed, installed setup tools and dependencies (that included installing the correct python version) and used the new python to install the package. This way, I got exposed to the possible errors we might encounter to. I also successfully troubleshooted the issue.

